

Threat Pilot

Agentic SBOM Risk Auditor

Architecture guide, file-by-file reference, tech-stack reasoning, judge questions and presentation script

Event	Hackatopia 2026, Cyber Security + Blockchain track
Problem statement	CB-02: Automated Software Bill of Materials (SBOM) Risk Auditor
One-line pitch	Scanners flag every CVE. Threat Pilot proves which ones your code can reach, ranks them by real risk, and opens a verified pull request after a human approves.
Stack	Python, FastAPI, LangGraph, Gemini or Ollama, Neon Postgres, React, TypeScript, Tailwind, shadcn/ui
State of the code	55 backend tests passing. Frontend type-checks and builds. Verified live against real OSV, EPSS and KEV data.

This document describes the code as committed. Where something is unproven or approximate it says so, because judges will ask.

1. The problem and the idea

A typical Python service pulls in 50 to 200 packages. A dependency scanner reports every known vulnerability (CVE) in every one of them, whether or not the application ever runs the affected code. Teams receive hundreds of alerts and cannot tell which three matter.

Severity is not risk. A critical CVE in a package that is never imported is less urgent than a medium CVE in a function that is called on user input. CVSS alone cannot express that difference.

What Threat Pilot does

1. **Builds the SBOM** for a GitHub repository: every direct and transitive package, from a real isolated install.
2. **Maps each package to live vulnerability data:** CVSS severity, EPSS exploit probability, CISA KEV (known exploited) status and fixed versions.
3. **Proves reachability:** for each vulnerability, is the package imported, and is the vulnerable function actually called? It reports the file and line.
4. **Ranks by real risk** with a transparent formula anyone can recompute by hand.
5. **Proposes a fix:** the smallest safe version upgrades, plus rewrites of functions that call vulnerable or deprecated APIs.
6. **Verifies the fix** before showing it, and loops back if the check fails.
7. **Waits for a human.** Nothing is written to GitHub until someone approves. Then it opens a pull request.

The result in one real run

On a public Streamlit project with a 59-package `requirements.txt`, a scanner-style count gives **133 vulnerabilities**. Threat Pilot found that **130 are in packages the code never imports**, 2 are in imported packages, and **1 is reachable**: `frontend/dashboard.py:76` calls `requests.post`, which reaches the vulnerable function through six calls inside the library. The proposed pull request upgrades two packages and flags one major-version bump for manual review.

2. Architecture at a glance

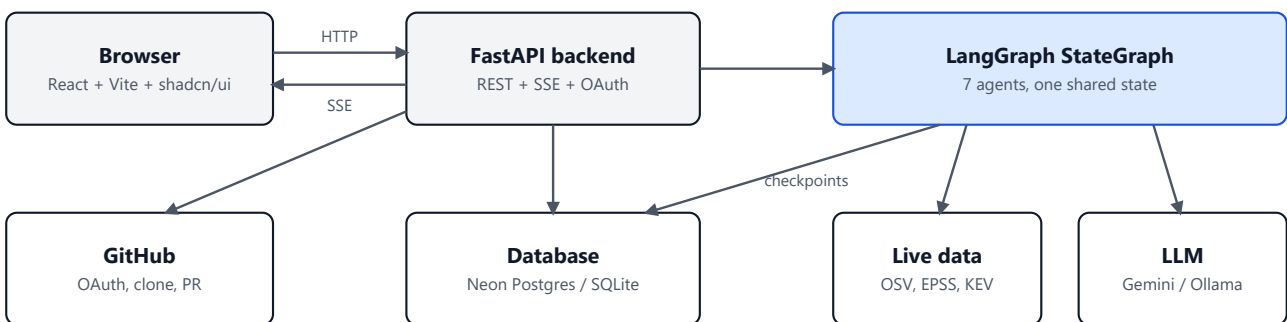


Figure 1. The four parts: a React frontend, a FastAPI backend, a LangGraph agent pipeline, and one database.

Part	What it is	Why it exists
Frontend	React single-page app. Pages: landing, dashboard, history, audit.	Lets a user pick a repo, watch the agents work, read the evidence and approve the fix.
API	FastAPI. REST endpoints, one Server-Sent Events stream per audit, GitHub OAuth.	Starts audits, streams progress, serves results, takes the approve / reject decision.

Part	What it is	Why it exists
Agent pipeline	A LangGraph StateGraph: 7 agents that share one typed state object.	Each agent owns one responsibility. They never call each other; they only read and write shared state.
Database	Neon PostgreSQL when DATABASE_URL is set, a local SQLite file otherwise.	Audits, logs, sign-in sessions, cache, per-repo memory and graph checkpoints.
External services	OSV.dev, FIRST EPSS, CISA KEV, GitHub, an LLM, optionally LangSmith.	Live facts, repository access, text understanding and tracing.

Design principle

Facts come from deterministic tools. The LLM only reads and writes text. Versions, CVE ids, scores and fixed versions are never produced by the model. The LLM is used in four places (reading advisory prose, writing explanations, writing PR text, proposing a function rewrite), and in each one code checks its output and has a fallback that works with no model at all.

3. The agent pipeline

Seven agents run in one graph. Two of them, Risk Analysis and Remediation, are made of two steps each (two graph nodes), which is how the parallel fan-out and the retry loop stay visible to LangGraph. The diagram shows the order, the loop and the three ways a run can end without a pull request.

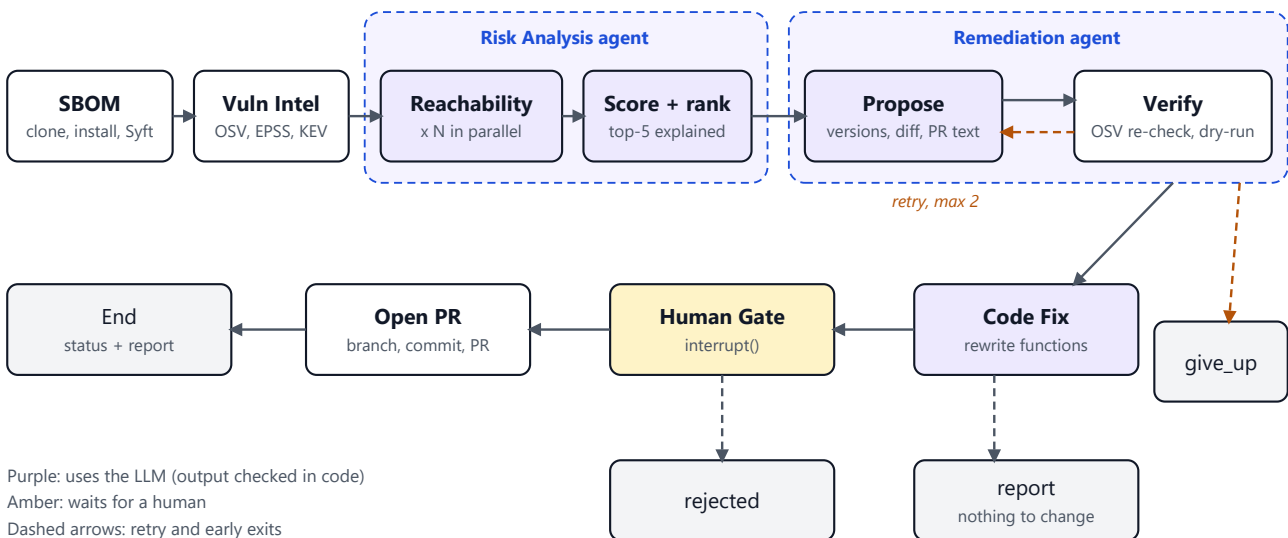


Figure 2. The LangGraph pipeline, defined in backend/app/graph.py.

#	Agent	LLM?	What it does	File
1	SBOM	No	Clones the repo, installs requirements.txt (and the files it includes) into an isolated virtualenv, runs Syft, marks each package direct or transitive.	agents/sbom.py
2	Vuln Intel	No	One batch query to OSV, fetches each advisory, merges duplicates, adds CVSS, EPSS and KEV, orders imported packages first.	agents/vuln_intel.py
3	Risk Analysis	Yes	Step 1, in parallel: one worker per vulnerability finds the vulnerable function, scans the app source and assigns L0, L1 or L2 with evidence. Step 2: computes the score, ranks everything, explains the top five and compares with the last audit of this repo.	agents/risk_analysis.py

#	Agent	LLM?	What it does	File
4	Remediation	Yes	Step 1, propose: chooses the smallest safe version per package, patches the manifest, builds the diff and PR text. Step 2, verify: re-queries OSV at the new versions and runs pip install --dry-run. A failed check loops back to step 1 (max 2 retries).	agents/remediation.py
5	Code Fix	Yes	Finds direct vulnerable calls and deprecated API calls, rewrites the enclosing function, keeps the rewrite only if it passes checks.	agents/code_fix.py
6	Human Gate	No	Pauses the graph with interrupt() until the user approves or rejects.	agents/human_gate.py
7	Open PR	No	Creates a branch, commits the changed files and opens the pull request through the GitHub API.	agents/open_pr.py

Why two of the agents have two steps. An agent here is a responsibility, not a single function. Risk Analysis answers "how much does each finding matter"; Remediation answers "what is a fix that actually works". Keeping their inner steps as separate graph nodes means LangGraph still runs the reachability workers in parallel and still checkpoints every pass of the retry loop.

The three LangGraph mechanisms that matter

- **Parallel fan-out.** `fan_out_reachability()` returns one `Send("risk_analysis_worker", ...)` per finding. Results merge back through an `operator.add` reducer on `reach_results`.
- **Critic loop.** Inside the Remediation agent, `route_after_verification()` sends a failed check back to the propose step with the rejected versions. After two retries it goes to `give_up` and the run ends with a report and no PR.
- **Human in the loop.** `interrupt()` in the Human Gate suspends the graph. State is checkpointed in the database, so the decision can arrive minutes later, or after a backend restart. The API resumes with `Command(resume={"approved": ...})`.

How one audit flows, end to end

1. The browser sends `POST /audit` with a repo URL. `runs.create_run()` creates a run and starts the graph in a worker thread.
2. The browser opens `GET /audit/{id}/stream`. Every agent emits running then done events, which arrive over SSE.
3. As each agent finishes, the browser refreshes `GET /audit/{id}` and the tabs fill in.
4. The graph reaches the Human Gate and pauses. The stream closes with status `awaiting_approval`.
5. The user reads the Fix tab and clicks Approve. The browser sends `POST /audit/{id}/decision`.
6. The graph resumes, Open PR runs with the approver's GitHub token, and the PR link appears.

4. Reachability levels and the risk score

L0, L1 and L2 are not kinds of vulnerability. They are three answers to one question: **how close does your code come to running the vulnerable code?**

Level	Meaning	How it is decided	Weight
L0	The package is never imported by the application.	No import of any of the package's modules anywhere in the repo.	0.2
L1	The package is imported, but the vulnerable function is not called.	An import is found; no call to the vulnerable function is. Evidence: the import line.	0.6

Level	Meaning	How it is decided	Weight
L2	The vulnerable function is called.	A call is found, directly or through the library's own code. Evidence: file, line and the call chain.	1.0

How a level is worked out

1. **Find the vulnerable function.** A small curated map is checked first (`data/curated_vuln_functions.json`). Otherwise the LLM reads the advisory and proposes names, and `packages.validate_functions()` discards any name not defined in the installed package.
2. **Scan the application.** `ast_scanner.scan()` parses every `.py` file with Python's `ast` module. It follows import aliases (`import yaml as y, from yaml import load as l`) and simple instances (`s = requests.Session(); s.get(...)`).
3. **Check for indirect calls.** If there is no direct call, `callgraph.entry_points()` builds a call graph of the installed package and finds which public functions reach the vulnerable one. If the app calls one of those, it is L2 and the chain is shown.
4. **Apply safe-pattern rules.** `yaml.load(f, Loader=yaml.SafeLoader)` is downgraded from L2 to L1.

The risk score

score = CVSS × reach weight × (1 + EPSS) × KEV boost × direct boost

- **CVSS** (0 to 10): how bad it is if exploited. Parsed from the vector string; falls back to the severity label.
- **Reach weight:** 0.2, 0.6 or 1.0 from the level above.
- **EPSS** (0 to 1): probability of exploitation in the next 30 days, from FIRST.
- **KEV boost:** 2.0 if the CVE is in CISA's Known Exploited Vulnerabilities catalogue, else 1.0.
- **Direct boost:** 1.2 if the package is listed in `requirements.txt`, else 1.0.

Finding	CVSS	Reach	1 + EPSS	KEV	Direct	Score
PyYAML, reachable (L2), direct	7.5	1.0	1.62	1.0	1.2	14.58
Pillow, never imported (L0), transitive	9.8	0.2	1.05	1.0	1.0	2.06

The CVSS 7.5 finding outranks the CVSS 9.8 one by a factor of seven. That reversal is the product. Every factor is shown in the UI so the number can be recomputed by hand.

What the levels do not prove

- **L0 is not "safe".** Another dependency may still use the package at runtime.
- **L2 is not "exploitable".** The code path exists; whether an attacker can feed it bad input is a separate question.
- **The analysis is static.** Calls through `getattr`, plugins or objects passed between functions are missed. The indirect chain is matched by name and can over-report.
- **Only .py files are scanned.** Notebooks and templates are not.

5. Tech stack and why

Layer	Choice	Why this	Why not the alternative
Agent orchestration	LangGraph 1.2	Gives the three things this problem needs as built-ins: parallel fan-out (Send), cycles for the critic loop, and interrupt() with checkpointing for human approval.	Plain LangChain chains have no cycles or pause/resume. CrewAI / AutoGen are conversation-driven; agents here should not chat, they should run a fixed, auditable pipeline. Hand-written asyncio means rebuilding checkpointing.
Language	Python 3.12	The target ecosystem is PyPI. Python's own ast module parses the code being audited, and pip resolves the dependencies.	Node or Go would need a third-party Python parser and could not reuse pip's resolver.
API	FastAPI + SSE	Async, typed with Pydantic, and Server-Sent Events are one-way streaming over plain HTTP, which is all a progress feed needs.	WebSockets add a second protocol for a one-way stream. Polling is laggy and wasteful. Flask has no native async.
SBOM	Syft (CycloneDX)	Industry-standard SBOM generator with a standard output format. pipdeptree is the fallback and supplies parent links.	pip freeze has no dependency tree. Parsing requirements.txt misses every transitive package.
Vulnerability data	OSV.dev + EPSS + CISA KEV	All free, no API key, and OSV has a batch endpoint and structured fixed-version ranges.	NVD is rate-limited and has no ecosystem-specific version ranges. Snyk / GitHub Advisory API need accounts or tokens.
LLM	Gemini 3.5 Flash (default), Ollama optional	Fast and cheap for short structured-output calls. One config string switches provider through init_chat_model.	A larger model adds latency and cost for tasks that are small and code-checked. Ollama is supported for offline or private use.
Database	Neon PostgreSQL, SQLite fallback	Serverless Postgres: no server to run, and LangGraph ships a Postgres checkpoint. SQLite keeps the app runnable with zero setup.	MongoDB has no first-party LangGraph checkpoint. Redis is not durable by default. Files do not survive scaling.
Tracing	LangSmith	Traces LangGraph natively: every node, tool and LLM call with inputs, outputs and timings, with one environment variable.	OpenTelemetry would need manual instrumentation for the same picture.
Frontend	React 18 + TypeScript + Vite	Typed contract shared with the backend schemas; Vite gives instant reloads during a 24-hour build.	Next.js adds server rendering this app does not need. Plain JS loses the typed API contract.
UI kit	Tailwind v4 + shadcn/ui	Accessible components copied into the repo, so they can be restyled freely. No runtime CSS-in-JS.	MUI / Ant are heavy and look generic. Hand-written CSS is slow to keep consistent.
Auth	GitHub OAuth, server-side sessions	The PR is opened as the real user. The token never reaches the browser; it is encrypted at rest.	A shared bot token hides who approved. JWT in localStorage exposes the token to scripts.

6. File-by-file reference: backend

All paths are under backend/app/. Line counts are approximate.

Core

File	Purpose	Main functions
main.py	The FastAPI app: every HTTP endpoint, the SSE stream, startup (logging, cleanup loop).	start_audit(), get_audit(), stream_audit(), decide(), list_audits(), audit_logs(), health(), lifespan()
graph.py	Wires the agents into a LangGraph StateGraph and compiles it with a database checkpoint.	build_graph(), fan_out_reachability(), give_up_node(), rejected_node(), report_node()
state.py	The shared state type every agent reads and writes.	class AuditState (TypedDict)
schemas.py	Pydantic models for the API. Mirrored by frontend/src/types.ts.	Component, Finding, Reachability, ScoredFinding, Remediation, CodeFix, AuditSnapshot, AuditSummary
runs.py	Run manager: runs the graph in a worker thread, forwards events to SSE subscribers, saves audits, restores a paused audit after a restart.	create_run(), resume_run(), snapshot(), revive(), history(), _execute()
events.py	Progress events. Wraps every agent so it emits running / done / failed and logs its duration.	emit(), traced()
auth.py	GitHub OAuth login and callback, sessions (token encrypted with Fernet), the user's repo list, which token a run uses.	login(), callback(), me(), repos(), session_of(), create_session(), current_run_token()
db.py	One storage interface over two engines. Tables: audits, logs, sessions, cache, memory.	class PostgresDatabase, class SqliteDatabase, get(), save_audit(), list_audits(), kv_get(), kv_put()
logs.py	Logging to console, a rotating file and the database. Tags each line with the audit's run id.	setup(), current_run_id()
config.py	Every setting, read from environment variables, with defaults.	MAX_FANOUT, MAX_FINDINGS, REACH_WEIGHT, DATABASE_URL, LLM_MODEL
stubs.py	Canned data so the whole pipeline runs with no network (USE_STUBS=true).	COMPONENTS, FINDINGS, REACH

Agents (agents/)

File	Purpose	Main functions
sbom.py	Thin node around tools/syft.py.	sbom_node()
vuln_intel.py	Queries, enriches and orders vulnerabilities.	vuln_intel_node(), imported_packages()
risk_analysis.py	Step 1: the per-finding worker that decides L0 / L1 / L2. Step 2: score, rank, explain, compare with the previous audit.	worker_node(), analyse(), vulnerable_functions(), rank_node(), compute_score(), since_last()
remediation.py	Step 1: version choice, manifest patch, diff and PR text. Step 2: the critic's two checks and the retry routing.	propose_node(), plan(), choose_version(), patch_manifests(), pr_text(), verify_node(), check_vulnerabilities(), dry_run(), route_after_verification()
code_fix.py	Finds and rewrites functions; accepts a rewrite only if it passes checks.	collect_issues(), enclosing_unit(), llm_rewrite(), rule_rewrite(), fix_file(), _accept()
human_gate.py	Pauses for the human decision.	human_gate_node(), route_after_gate()
open_pr.py	Collects the changed files and calls GitHub.	open_pr_node()

Tools (tools/)

File	Purpose	Main functions
syft.py	Workspace and SBOM: clone, virtualenv, install, Syft, pipdeptree fallback, repository-dependency detection, 30-day cleanup.	prepare_workspace(), generate_components(), components_for(), repository_requirements(), cleanup_workspaces()
osv.py	OSV.dev client: batch query, advisory fetch, fixed versions, duplicate merging.	query_batch(), get_vuln(), fixed_versions(), to_finding(), dedupe()
epss.py / kev.py	EPSS probabilities and the CISA KEV catalogue.	epss_scores(), kev_set()
cvss.py	CVSS vector to numeric score, with a label fallback.	base_score(), score_vector()
ast_scanner.py	Parses the application source: imports, calls, aliases, instances, safe patterns.	scan(), scan_file(), imported_roots(), class _FileScanner
callgraph.py	Static call graph of an installed package, for indirect reachability.	build(), entry_points(), path_for(), class Graph
packages.py	Inspects installed packages: import names and defined symbols.	import_names(), defined_symbols(), validate_functions()
deprecations.py	Discovers deprecated APIs from installed source and the standard library.	class Index, for_package(), for_stdlib()
llm.py	The only LLM entry point: provider selection, cached structured-output calls, a circuit breaker, and the four call sites.	get_llm(), structured(), extract_functions(), explain_findings(), draft_pr_text(), rewrite_function()
github.py	GitHub REST calls: branch, commit, PR, and a fork when the user cannot push.	create_branch_commit_pr()
versions.py	Version comparison helpers.	highest(), newer_than(), is_major_bump(), normalize()
cache.py	Database-backed cache for external and LLM responses.	get(), put(), cached()
textio.py	Reads and writes files without changing their line endings.	read_source(), to_source()

Tests (backend/tests/) - 55 tests, no network needed

File	What it proves
test_pipeline.py	The real agents end to end on a fake repo with the network faked: reachable finding outranks higher CVSS, critic loop recovers, reject writes nothing, line endings preserved.
test_scanner.py	Alias handling, SafeLoader downgrade, other packages with the same function name are ignored.
test_remediation.py	Version choice, major bumps to manual review, manifest patching, duplicate merging.
test_code_fix.py	Rewrites accepted or rejected by the checks, instance method resolution, the call graph.
test_persistence.py	An audit survives a restart and can be approved, per-repo memory, logs, session encryption, workspace cleanup, deprecation discovery.
test_auth.py / test_api.py	OAuth round trip, forged state and open redirect rejected, the HTTP flow and SSE stream.
test_llm.py / test_graph.py / test_risk_score.py	Provider selection, the stub graph, the score ordering.

7. File-by-file reference: frontend

All paths are under frontend/src/.

File	Purpose
main.tsx / App.tsx	Entry point and routes: / (landing), /dashboard, /history, /audit/:runId.
types.ts	TypeScript mirror of the backend schemas. Change it together with schemas.py.
api/client.ts	Every backend call, the SSE subscription, and a mock mode (VITE_USE MOCKS=true) that runs the UI with no backend.
hooks/useAudit.ts	Follows one audit: opens the stream, collects events, refreshes the snapshot, sends the decision.
hooks/useAuth.ts	Who is signed in, their repositories, and sign-in errors returned by the OAuth callback.
pages/Landing.tsx	Hero section and the five-step explanation.
pages/Dashboard.tsx	The user's repositories with their latest audit, import by URL, recent audits.
pages/History.tsx	Every audit, filterable by status and repository.
pages/AuditPage.tsx	One audit, as tabs: Overview, Findings, Fix, SBOM, Activity, History.
components/Overview.tsx	The headline number, key figures, the review call-to-action and the top risks.
components/AuditCharts.tsx	Three charts: alerts-to-action funnel, reachability breakdown, findings per package.
components/RiskTable.tsx + EvidenceRow.tsx	The findings table with risk vs CVSS ordering, and the expandable evidence: file and line, call chain, score formula.
components/ApprovalPanel.tsx	The Fix tab: upgrades, diff, code changes, verifier notes, PR preview, Approve and Reject.
components/CodeFixes.tsx / DiffViewer.tsx / PrBody.tsx	Rewritten functions per file, a unified-diff renderer, and a small Markdown renderer for the PR preview.
components/PipelineStrip.tsx / AgentTimeline.tsx	The one-line pipeline status, and the detailed per-agent timeline with parallel workers.
components/LogsPanel.tsx / RepoHistory.tsx	The backend log for this audit, and all audits of the same repository.
components/ComponentsPanel.tsx	The SBOM as a dependency tree.
components/Header.tsx / GlassDock.tsx / RunBanner.tsx / UrlForm.tsx	Top bar with the glass dock navigation, the audit's status banner, the repository URL form.
components/ui/*	shadcn/ui components (button, card, table, tabs, accordion, tooltip and others).
lib/reach.ts / format.ts / brand.ts	Level colours and labels, time formatting, the product name.
mocks/mockAudit.ts	Realistic sample data for mock mode.

8. How the main functions connect

Reading top to bottom follows one audit. Indentation means "is called by the line above".

```
main.start_audit()
  runs.create_run() - starts runs._execute() as a background task
    graph.stream() in a worker thread; each node is wrapped by events.traced()
```

```

sbom_node() - syft.components_for()
    syft.prepare_workspace() - git clone, venv, pip install
    syft.generate_components() - Syft, pipdeptree
vuln_intel_node()
    osv.query_batch(), osv.get_vuln(), osv.to_finding(), osv.dedupe()
    epss.epss_scores(), kev.kev_set(), imported_packages()
risk_analysis agent
    fan_out_reachability() - one Send per finding to worker_node() - analyse()
    vulnerable_functions() - curated map, else llm.extract_functions() +
    packages.validate_functions()
    ast_scanner.scan() - direct calls; callgraph.entry_points() - indirect calls
rank_node() - compute_score(), llm.explain_findings(), since_last()
remediation agent
    propose_node() - plan(), choose_version(), patch_manifests(), pr_text()
    verify_node() - check_vulnerabilities(), check_resolution() - dry_run()
    route_after_verification() - back to propose_node(), or on
code_fix_node() - collect_issues(), fix_file()
    llm_rewrite() or rule_rewrite(), then _accept()
human_gate_node() - interrupt(); the graph pauses here
main.decide() - runs.resume_run() (or runs.revive() after a restart)
    open_pr_node() - github.create_branch_commit_pr()
runs._persist() - db.save_audit(), per-repo memory

```

9. Data, storage and API

API endpoints

Method	Path	Purpose
POST	/audit	Start an audit for a GitHub repo URL. Returns run_id.
GET	/audit/{id}/stream	Server-Sent Events: agent progress. Replays past events first.
GET	/audit/{id}	The full snapshot: components, scored findings, remediation, code fixes, verification, events.
POST	/audit/{id}/decision	Approve or reject. Resumes the graph. Restores from a checkpoint after a restart.
GET	/audit/{id}/logs	The backend log for one audit.
GET	/audits	Audit history, newest first. Optional ?repo_url= filter.
GET	/auth/github/login, /callback, /me, /repos; POST /auth/logout	GitHub OAuth and the signed-in user's repositories.
GET	/health	Storage engine, LLM model, SBOM tool, tracing on or off.

Database tables

Table	Holds
audits	One row per audit: status, who ran it, a summary for the history list, and the full snapshot as JSON.
logs	Every log line, tagged with run id and agent.
sessions	Sign-in sessions. The session id is stored hashed; the GitHub token is encrypted with Fernet.
cache	Responses from OSV, EPSS, KEV, pip dry-runs, deprecation scans and LLM calls.
memory	Per repository: the finding ids of its last audit, so the next audit reports what is new or resolved.
checkpoint tables	Created by LangGraph. Graph state after every step, keyed by run id.

Trust and safety design

- **Human approval is mandatory.** Open PR is the only node that writes to GitHub and it sits behind the interrupt.
- **LLM output never becomes a fact unchecked.** Function names are validated against package source; explanations and PR text are rejected if they cite an id the finding does not have; rewrites must parse, keep the signature and remove the flagged call.
- **No major-version bumps automatically.** They are listed for manual review.
- **The fix is verified before it is offered.** OSV is re-queried at the new versions and the manifest must resolve.
- **Tokens stay on the server.** The browser holds an HttpOnly cookie. Tokens are stripped from logs, traces and subprocess output.
- **Target code is isolated.** Each repo is cloned and installed in its own directory and virtualenv, deleted 30 days after its last audit.
- **Input is validated.** Only https://github.com/owner/repo URLs are accepted, which also blocks git argument injection.

Known limitations - say these before a judge finds them

- Python and requirements.txt only. No npm, Maven, Poetry or pyproject.toml.
- Reachability is static and name-based for the indirect chain. It can miss dynamic calls and can over-report.
- Installing a target repo's requirements runs that project's install scripts. Isolation is a virtualenv, not a sandbox.
- Rewritten functions are checked for syntax, signature and the flagged call. They are not run against the project's tests.
- Old pins sometimes have no prebuilt wheel for the backend's Python version, so they fail to install; the SBOM then falls back to the manifest for those packages.
- Runs in progress are lost on restart (paused ones are not). There is one backend process; no horizontal scaling.
- No rate limiting or per-user access control on audits. Anyone with the URL can read an audit.

10. Questions judges are likely to ask

About the idea

How is this different from Dependabot, Snyk or pip-audit?

Those tools answer "which packages have known CVEs". We answer "which of those can your code actually reach", with file-and-line evidence, and rank by that. pip-audit and Dependabot would raise all 133 alerts on our test repo; we show that 1 is reachable. Snyk has reachability for some languages as a paid feature; ours is open, explainable and shows the call chain.

Why should I trust the reachability result?

Every L1 and L2 result comes with the file and line, and for indirect calls the chain of functions. Nothing is asserted without evidence you can click. It is static analysis, so we say plainly that L0 means "not imported by your code", not "safe", and that the indirect chain is evidence for review, not a proof.

Why do you still report L0 findings?

Because unreachable is not the same as harmless: another dependency may use the package. We report them at low weight and do not patch them automatically, unless the CVE is in CISA KEV, in which case it is patched regardless.

Who is the user?

A developer who wants the three things to fix today, a security engineer who wants evidence instead of guesses, and an engineering lead who will not let a bot change production without a check and an approval.

About the agents and the LLM

Why is this "agentic"? Could it not be a script?

Three parts genuinely need agent orchestration: parallel workers that fan out per vulnerability, a critic step that rejects a fix and sends it back, and a pause for a human that can last minutes and survive a restart. LangGraph gives those as first-class features with checkpointed state. The honest answer is that most steps are deterministic on purpose: that is what makes the result reproducible.

What exactly does the LLM do, and what if it hallucinates?

Four things: read an advisory to name the vulnerable function, write a two-sentence explanation, write PR text, and propose a function rewrite. Each is checked in code. A function name not defined in the installed package is dropped. Text that cites a CVE the finding does not carry is replaced by a template. A rewrite that does not parse, changes the signature or still makes the flagged call is discarded. With no LLM at all, the whole pipeline still runs.

What happens if the LLM or an API is down?

The LLM has a circuit breaker and deterministic fallbacks. External responses are cached in the database, and OFFLINE=true runs from cache only.

How does the verifier loop work?

It re-queries OSV at each proposed version and runs pip install --dry-run on the patched manifest. If a vulnerability with an available fix remains, or the manifest does not resolve, it records the rejected version and a suggested one and routes back to Remediation. After two retries it gives up and the run ends with a report and no PR.

Why LangGraph and not CrewAI or AutoGen?

Those frameworks centre on agents talking to each other. We wanted a fixed graph with typed shared state, cycles, parallel branches and a durable pause. LangGraph models exactly that, and ships Postgres checkpointing.

About correctness and security

How is the risk score justified? Are the weights arbitrary?

The inputs are standard: CVSS for impact, EPSS for likelihood, KEV for confirmed exploitation. The reach weights (0.2, 0.6, 1.0) and boosts are our judgment and are configurable. What matters is that the formula is shown and every factor is visible, so anyone can disagree with a weight and recompute.

Is it safe to clone and install someone's repository?

It runs in its own directory and virtualenv, but pip install does execute that project's build scripts, so this is isolation, not a sandbox. In production it belongs in a container or microVM with no credentials and restricted network. We say that openly.

Could the tool open a harmful pull request?

It cannot open anything without approval. Before approval the user sees the exact diff and PR text. Major-version bumps are never automatic, and the upgrade is verified. Rewritten code is clearly labelled as not run against tests.

How do you handle the GitHub token?

OAuth with a server-side session. The browser only has an HttpOnly cookie. The token is encrypted at rest, never placed in graph state, and scrubbed from logs, traces and command output.

How did you test this?

55 automated tests that run with no network, including the full pipeline on a fake repo, the critic loop, restart-and-resume, and the OAuth flow with forged-state and open-redirect cases. Plus live runs against real OSV, EPSS and KEV data on a public repository.

About scale and what is next

Does it scale to a large monorepo?

The cheap scan runs on every finding; the LLM step is capped at 30 findings and rewrites at 8 functions per audit. Results are cached. The limits today are one backend process and Python-only. Scaling out means a job queue and workers, which the checkpointed design already allows.

How would you support npm or Java?

The pipeline is ecosystem-neutral after the SBOM. Each ecosystem needs three adapters: an installer, a source parser (tree-sitter for JavaScript) and its OSV ecosystem name.

Where is the blockchain part of the track?

Not built. The natural extension is signing each SBOM and audit result and anchoring the hash on-chain, so anyone can verify what was audited and when. We chose depth on the security problem over a thin blockchain layer.

What would you do with another month?

A GitHub Action that comments the risk change on every pull request, sandboxed installs, running the project's tests against the fix, and JavaScript support.

11. Presentation: script and tips

Structure for a 5-minute slot

Time	Section	Goal
0:00 - 0:40	The problem	Make them feel alert fatigue with one number.
0:40 - 1:10	The idea	Severity is not risk. Reachability is the missing signal.
1:10 - 3:40	Live demo	One repo, start to pull request.
3:40 - 4:30	How it works	The pipeline diagram and the trust design.
4:30 - 5:00	Honest limits and what is next	Show you know the edges.

Script

Opening (problem)

"This is a small Python project. Fifty-nine packages. If I run a normal dependency scanner on it, I get one hundred and thirty-three security alerts. Nobody fixes one hundred and thirty-three alerts. So nobody fixes any. That is the real vulnerability."

The idea

"The scanner is not wrong. It is answering the wrong question. It tells you the vulnerable code is on your disk. It does not tell you whether your program ever runs it. Threat Pilot answers that second question, and it shows its evidence."

Demo, step 1: start

"I sign in with GitHub, pick a repository, and start an audit. These are the agents. Each one has a single job, and you can watch them work."

Demo, step 2: the answer

"One of one hundred and thirty-three. One alert is in code this application actually reaches. Here is the funnel: all alerts, the ones in packages we import, the one we can reach."

Demo, step 3: the evidence

"I do not have to take its word for it. This is the file and the line. Our code calls requests.post, and this is the chain of calls inside the library that ends at the vulnerable function. And this is the score, factor by factor. You can recompute it by hand."

Demo, step 4: the fix and the gate

"It has already prepared a fix: two upgrades, checked against the vulnerability database and against pip. One upgrade needed a major version, so it refused to do that automatically and flagged it for me. Nothing has touched GitHub yet. I approve, and here is the pull request."

How it works

"Under the hood it is a LangGraph pipeline. Three things make it more than a script: the reachability workers run in parallel, a verifier can reject a fix and send it back, and the graph pauses for a human and survives a restart. The language model only reads and writes text. Every fact comes from a deterministic tool, and everything the model produces is checked in code."

Close

"It is Python-only today, and reachability is static analysis, so it is evidence, not proof. But it turns one hundred and thirty-three alerts into one decision, with the reasoning attached. Thank you."

Demo preparation

- **Use a repository you own** so the pull request lands in your account, and confirm one real approval works end to end before the day.
- **Warm the cache.** Run the demo audit once beforehand; the second run skips the install and most network calls.
- **Plant a clear L2.** A repo with `PyYAML==5.3` and a `yaml.load(f)` call gives a direct L2 and a function rewrite. Check it installs on your machine's Python first.
- **Show the critic loop on purpose.** Set `FORCE_BAD_VERSION=pyyaml==5.3.1` so the verifier visibly rejects the first attempt and recovers.
- **Have a fallback.** If the network fails: `OFFLINE=true` with a warm cache, or the frontend's mock mode, or a recorded video. Decide the order in advance.
- **Keep the LangSmith trace open in another tab** for the "how do the agents actually run" question.

Delivery tips

- Lead with the number (133 to 1), not the architecture. Architecture is the answer to "how", which comes after "why should I care".
- Say "evidence, not proof" yourself. Naming a limitation first reads as rigour; having it found reads as a gap.
- When asked something you did not build, say so in one sentence and then say how the design would take it. Do not improvise a feature.
- One person drives the demo, one person talks. Never narrate while typing.
- Do not read the screen. Point at one thing per screen: the headline number, the file and line, the Approve button.
- If the demo breaks, keep talking and switch to the fallback without apologising. Judges remember recovery.
- Rehearse twice with a timer. Cut the script before you cut the demo.

Appendix: what we deliberately did not build

Each of these is a reasonable-sounding idea that would add complexity without making the core claim ("1 of 133 is reachable") more true. Useful when a judge asks "why didn't you...".

Idea	Why it would be over-engineering here
Microservices: one service per agent, with a message queue	The agents already share state through LangGraph. Splitting them adds network hops, deployment and failure modes for a pipeline that runs in one process in about a minute.
Kubernetes, autoscaling, service mesh	There is one backend process and one user at a time. A container and a managed database are enough.
A vector database and RAG over all advisories	Advisories are looked up by exact package and version through OSV. There is nothing to search semantically.
Agents that debate or vote on each finding	Reachability is a factual question with a deterministic answer. More model calls would add cost and variance, not accuracy.
Fine-tuning a model on CVE data	The model does four small text tasks that are checked in code. A general model with validation is sufficient.

Idea	Why it would be over-engineering here
Full inter-procedural data-flow and taint analysis	It would take months and still not be sound for Python. Import, call and call-chain evidence already changes the ranking.
A blockchain ledger for every audit event	Signing the final SBOM and anchoring one hash would cover verifiability. Recording every event on-chain adds cost and nothing else.
A custom graph database for dependencies	A dependency tree of a few hundred nodes fits in a JSON column.
Multi-tenant organisations, roles and billing	Out of scope for proving the idea. GitHub identity is enough for a demo.
A plugin system for scoring formulas	The weights are five numbers in a config file.